

CS439 Project Checkpoint 1 – Extended Project Proposal

Yahav Mangel, Siddharth Jain

Project Description

Portable devices such as wireless headphones, mice, and IoT sensors rely on WiFi or Bluetooth/BLE for communication. WiFi offers high throughput but consumes more energy, while Bluetooth/BLE provide lower energy usage but at reduced throughput. Current systems often stick to one communication interface/protocol and do not intelligently switch between them. However, works such as [2] prove that there is potential for improved performance when using more than one interface/protocol at a time.

This project proposes to recreate and expand on an **application-level protocol switching algorithm** that dynamically switches between WiFi and BLE depending on observed data stream characteristics. The algorithm will monitor outgoing data telemetry – such as packet size, urgency, and type – then select the most suitable interface to balance throughput and power efficiency. We will initially aim to create a similar setup and measurement system to [1], and then expand with other metrics and compare their performance.

The goal of this project is to maximize throughput while minimizing energy consumption and improving the overall user experience for portable devices such as cellular phones, wireless mice, headphones, and IoT nodes. High power consumption and degraded user experience – particularly due to persistent network connectivity requirements and mobility-related challenges – have long been recognized issues. Prior work such as [3], [4], and [5] has explored these problems and proposed various solutions. Building on these efforts, we believe this project will contribute meaningfully to addressing these challenges.

Motivation

Battery-powered IoT devices face a fundamental tradeoff: WiFi provides high throughput but consumes significant power, while BLE is power-efficient but offers limited bandwidth. Current devices commit to one protocol at design time, forcing designers to sacrifice either performance or battery life. Dynamic switching could achieve both goals by selecting protocols based on real-time workload characteristics, such as using BLE for small sensor updates and WiFi for large data transfers.

Related Works and Limitations

The Blue-Fi system [1] demonstrated BLE/WiFi coexistence on mobile devices, showing intelligent switching reduces power while maintaining quality of service. However, Blue-Fi targeted mobile architectures with separate radio chips rather than integrated IoT microcontrollers. More recently, Mao et al. [6] applied reinforcement learning to adaptive video streaming, showing RL agents can discover effective scheduling policies that outperform hand-tuned rules.

Despite these successful, strongly related solutions, these and other related works have the following common limitations:

- They often use fixed metrics to make switching decisions.
- They often target different hardware.
- They often require manual tuning for best results.
- They often fail to take advantage of state-of-the-art RL-based algorithms, especially in the case of resource-constrained devices.

Activities

As we progress through this project, we plan to progress in the following manner:

- Literature review on cross-technology communication and interface scheduling: Done!
- Set up hardware and Arduino IDE to demonstrate basic protocol functionality: Done!
- Implement protocol switching logic at the application layer: In Progress, finish by Checkpoint 2
- Design switching policies (heuristic-based, as in [1], then lightweight ML-based): In Progress, finish by Checkpoint 2
- Develop monitoring software to capture traffic characteristics: Finish before final presentation
- Test on use cases: audio streaming, mouse input, sensor data: Finish before final presentation
- Evaluate power consumption vs throughput: Finish before final presentation

Expected Results

After progressing with the above items, we hope to produce the following as output:

- A prototype demonstrating reading, manipulating, and seamless switching between WiFi and Bluetooth/ZigBee traffic.
- Quantitative results showing power savings without noticeable performance loss.
- Comparison with single-technology baselines (standalone BLE and Wi-Fi) in standard wireless data delivery.

Equipment and Resources

We are utilizing the following list of equipment for our experimental hardware setup:

- 4 × ESP32 Microcontrollers
- 2 × Breadboards
- 10 × Jumper cables
- 4 × USB micro -> USB-C adapter cables
- Power measurement tools (multimeter, USB power monitor, or profiling software)

Project Plan

- Weeks 1–2: Literature/code/schematic review
- Week 3–4: Hardware setup (Feather + XBee)
- **By Checkpoint 1:** Be able to read and manipulate WiFi/BLE/ZigBee data, measure power consumption
- Week 5: Implement protocol switching logic
- Week 6: Develop traffic monitoring/reading software
- Week 7: Write test cases + optimize algorithm

- Week 8: Collect performance and power data
- **By Checkpoint 2:** Have switching algorithm + test cases implemented
- Week 9: Finalize data collection and generate report and presentation

Division of Work

- Initial research and hardware setup: Both
- Wi-Fi and BLE/ZigBee specifics/testing: split this up down the line
- Power Management and Measurement: Siddharth
- Algorithm Development and Testing: Both
- Data Analysis: Yahav
- Report/Presentation: Both

Progress So Far

In the following sections, we will elaborate in depth on our progress as of Checkpoint 1:

Hardware Setup (Weeks 1-2)

Both ESP32 boards are operational and programmable. We initially encountered an issue where one micro-USB cable was power-only rather than data-capable, delaying dual-board development. Both boards are now accessible and we've verified they can advertise BLE and WiFi services using scanner applications. We explored XBee modules for Zigbee but decided the dual-chip complexity was unnecessary given our limited microcontroller experience.

Development Environment (Weeks 1-2)

We configured two complementary workflows: Arduino IDE for firmware development and Python/VS Code for serial monitoring. macOS setup required installing CP210x drivers, modifying security settings for unsigned drivers, and granting USB access permissions. Both environments now reliably program ESP32 boards and monitor serial output.

Research (Weeks 1-3)

Completed thorough review of Blue-Fi [1] and Mao et al. [6] papers. Key insights include the importance of TDMA scheduling to prevent radio interference, power savings through intelligent protocol selection, and how RL actor-critic architectures suit continuous decision-making. Based on this research, we chose RL-based switching over simple threshold approaches.

Current Status

We have working hardware, configured development environment, and research foundation. We have also successfully established point-to-point communication between 2 hosts, using both Wi-Fi and BLE. We achieved this by using example code snippets from the Arduino IDE ESP32 development kit.

Currently, our biggest challenge is memory and buffer space limitations on our ESP32 chip. In particular, when we flash the ESP32 with code that imports both the Wi-Fi and BLE C++ libraries, we are unable to

complete the code import because the chip we are using does not have enough memory to accommodate both stacks at once. After discussing with the professor, we decided to order a better ESP32 which has enough space to accommodate our project needs. Thus, we will need to spend more time on researching the ML models as well as power data collection before we are able to proceed with the physical setup, as we are waiting for the new ESP32 to arrive.

Changes to Project Plan

In the following sections, we will elaborate in depth on the challenges and resulting changes we have made to the project, as of Checkpoint 1:

Scope Simplification: Testing

Original: Conduct all testing with mouse input, audio streaming, and sensor data.

Revised: After doing more research, we realized that mouse, audio, and sensor inputs may be too complex for initial testing, as it requires modification of underlying HID protocols. Rather than try to get these working early on, we will focus on simpler input data formats (such as text files) before tackling complex use cases. We'll establish reliable switching with basic inputs first, then incrementally add realistic workloads.

Scope Simplification: Protocols

Original: Unsure if to use BLE, ZigBee, or both in tandem with Wi-Fi.

Revised: After working through the hardware setup, we determined that we prefer to stick to BLE only (in tandem with Wi-Fi) and not incorporate ZigBee in any capacity. This is primarily due to the fact that we found the development environment for BLE to be much easier to use than ZigBee's: the Arduino IDE is very user friendly, and comes pre-packaged with many code examples for both protocols which demonstrate the use of the C++ APIs for both protocols in an ESP32-compatible manner! In contrast, there is less information available for both the Adafruit hardware running ZigBee, as well as the ZigBee protocol itself.

Timeline Modifications

Since the initial Project Proposal, we have not made many modifications to our project timeline. The only major changes we have made are: (1) spending more time than anticipated on the hardware setup and research (an additional week), and (2) we now plan to tackle the data collection frameworks (power and throughput measurement) later on, as those couple better with the development of the testing framework.

Both of these changes are reflected in the revised "Project Plan" section above.

Conclusion

We have established solid foundations despite initial hardware and environment challenges. Our platform is operational, the development workflow is functional, and research informs our implementation approach. The simplified scope in both testing and protocols provide a more realistic timeline and methodology for project execution. We are positioned to begin implementing and testing protocol switching in the coming weeks, with both practical rule-based and research-oriented RL approaches planned.

References

- [1] G. Ananthanarayanan and I. Stoica. Blue-Fi: Enhancing Wi-Fi Performance Using Bluetooth Signals. MobiSys 2009.
- [2] K. Chebrolu and A. Rao. Bandwidth aggregation for real-time applications in heterogeneous wireless networks. IEEE Transactions on Mobile Computing, 2006.
- [3] A. Rahmati and L. Zhong. Context-for-wireless: Context-sensitive energy-efficient wireless data transfer. MobiSys 2007.
- [4] M. Kassar, B. Kervella, and G. Pujolle. An overview of vertical handover decision strategies in heterogeneous wireless networks. Computer Communications, 2008.
- [5] S. Yin, S. Zhang, and Q. Li. Energy-efficient wireless communication for IoT: A survey. IEEE Communications Surveys & Tutorials, 2020.
- [6] H. Mao, R. Netravali, M. Alizadeh. Neural adaptive video streaming with Pensieve. SIGCOMM 2017.